

Razvoj web aplikacija

Projekt 1 - Backend

mag.ing. Josip Torić (SIT UNIZD)

05.10.2026.



Studij informacijske tehnologije
Sveučilišta u Zadru

Plan prezentacije

- 1 Što gradite i kako se boduje
- 2 Detaljni zahtjevi za backend
- 3 Bodovna raspodjela 1. dijela
- 4 Instalacija razvojnog okruženja
- 5 GitHub Education, Copilot i Claude Code
- 6 Kako koristiti AI da stvarno naučite
- 7 Što vas čeka na 1. obrani

Projekt kroz semestar

Tijekom semestra izrađujete **vlastitu full-stack web aplikaciju**. Tema je vaš izbor, ali mora zadovoljiti zahtjeve iz ove prezentacije.

Dio	Što se predaje	Obrana	Bodovi
1. dio	Backend: API, baza, testovi	16.11.	45
2. dio	Frontend SPA + deploy	18.01.	45
Dolasci na predavanja			10
Ukupno			100

Svaki dio se brani zasebno, usmeno, uz demonstraciju i pitanja o kodu. Danas govorimo o 1. dijelu - zahtjevi za frontend dolaze u prezentaciji „Projekt 2” na početku frontend bloka.

Tema projekta: kako odabrati

Dobra tema ima **uloge, pravila i statuse** - ne samo tablice koje se pune i čitaju.

- Najmanje dvije role s različitim ovlastima: admin i korisnik, organizator i sudionik, trener i sportaš
- Podaci koji pripadaju nekome - temelj za ownership provjere
- Barem jedno pravilo koje ovisi o vremenu, statusu ili kapacitetu
- Realan proces koji možete objasniti u dvije rečenice

Ako imate samo CRUD bez pravila i rola - to nije dovoljno za prolaz. Ako niste sigurni je li tema dovoljna, pitajte me **prije** nego što počnete.

Primjeri tema

- Sustav rezervacija termina
- Event management s prijavama
- Task / workflow sustav
- Coach & athlete tracker
- Document approval sustav
- Mini marketplace s oglasima
- Evidencija članstva u klubu
- Sustav narudžbi i dostave

Referentni primjer s predavanja je sustav prijava na powerlifting natjecanje: admin i klubovi, natjecatelji, rokovi nominacija. Vaša tema mora imati usporedivu složenost.

Tehnologije

Koristimo konkretan stack - isti za sve studente. Na predavanjima učimo svaku tehnologiju od nule.

Tehnologija	Uloga
Python 3.11+	jezik backenda
FastAPI	web framework
SQLAlchemy 2.0	ORM, async
Alembic	migracije baze
PostgreSQL	baza podataka
Pydantic	validacija i DTO
pytest	testovi
uv	paketi i virtualna okruženja
Docker Compose	baza u kontejneru

Drugi stack nije dopušten - cilj kolegija je dubina na jednom stacku, ne širina na

Struktura repozitorija

Projekt je **monorepo**: backend i frontend u istom repozitoriju na GitHubu.

Folder	Sadržaj
api/routers/	HTTP endpointi, bez poslovne logike
api/services/	poslovna logika i provjere
api/repositories/	svi upiti prema bazi
api/models/	SQLAlchemy ORM definicije
api/schemas/	Pydantic DTO modeli
api/core/	config, session, auth, errori
api/tests/	pytest testovi
web/	frontend - dolazi u 2. dijelu

Arhitektura: slojevi su obavezni

Routers → Services → Repositories → Models → PostgreSQL

Svaki sloj ima jednu odgovornost:

- Router prima request, parsira parametre, poziva service, vraća response
- Service sadrži poslovnu logiku - ne zna za HTTP ni SQL
- Repository radi sve upite prema bazi i vraća model objekte

Ako vam je sva logika u routeru ili router direktno piše SQL - to se mora popraviti **prije** obrane. Arhitektura nije preporuka, nego uvjet.

Zahtjev 1: baza podataka i migracije

- **Najmanje 3 tablice** s relacijama: 1:N obavezno, M:N ako tema traži
- Svaka tablica ima surrogate id kao primarni ključ
- **Constrainti na razini baze:** NOT NULL za obavezna polja, UNIQUE gdje domena traži jedinstvenost, FOREIGN KEY za sve veze
- Sve promjene sheme kroz **Alembic migracije** - ručni SQL na bazi nije dopušten
- Svaka migracija ima funkcionalan downgrade()
- **Seed skripta:** idempotentna, kreira role i demo podatke za obranu

Na obrani provjeravam: „Zašto je ovaj stupac UNIQUE? Što se dogodi ako constraint makneš i dva requesta stignu istovremeno?”

Zahtjev 2: autentikacija

- Login s username + password
- Lozinke hashirane **bcryptom** - plaintext ili MD5/SHA su nula bodova za autentikaciju
- **JWT access token** (15 min) + **refresh token** (dulji rok)
- Tri endpointa: /auth/login, /auth/refresh, /auth/me
- Kriva lozinka i nepostojeći username vraćaju **isti** 401 error - bez otkrivanja koji korisnici postoje
- JWT_SECRET iz env varijable, nikad hardkodiran u kodu

JWT payload sadrži samo ono što autorizacija treba: sub, role, vlasnički ID, exp. Nikad lozinku ni osobne podatke.

Zahtjev 3: autorizacija i ownership

- **Najmanje dvije role** s različitim ovlastima, npr. admin i korisnik
- Zaštita rola kroz dependency (`require_role`), ne if-ovima po routerima
- **Ownership**: korisnik vidi i mijenja **samo svoje** resurse - provjera u service layeru
- Ispravni status kodovi: 401 bez tokena, 403 s tokenom ali bez prava

Scenarij	Status	Zašto
Nema tokena ili istekao	401	Ne znamo tko si
Kriva rola	403	Znamo tko si, ne smiješ
Tuđi resurs	403 ili 404	Ovisno o politici otkrivanja

Na obrani mijenjam ID u URL-u i očekujem da vaš backend to zabrani - to je IDOR test.

Zahtjev 4: poslovna pravila

Barem **jedno pravilo koje nije CRUD** - nešto što service provjerava prije nego što dopusti akciju:

Tip pravila	Primjer
Rok	Prijava moguća samo do datuma X
Status / tranzicija	Zahtjev ide draft → poslan → odobren, ne preskače korake
Kapacitet	Termin prima najviše N rezervacija
Jedinstvenost u kontekstu	Isti korisnik se ne može dvaput prijaviti na isti događaj

Pravilo živi u **service layeru** i vraća domenski error s jasnim kodom, npr. `deadline_passed` uz status 400 ili 409. UI smije sakriti gumb, ali backend **mora** zabraniti.

Zahtjev 5: API surface

Najmanje **10-15 endpointova**, organiziranih oko resursa:

Grupa	Primjer	Min.
Auth	login, refresh, me	3
Glavni entitet	CRUD: list, create, get, update, delete	4-5
Drugi entitet	CRUD s ownership provjerom	4-5
Workflow	akcije: odobri, odbij, odjavi, zaključaj	1-2

- **Nested URL-ovi** gdje ima smisla: `/clubs/{id}/lifters`, ne `/lifters?club_id=5`
- Lista endpointova ima **paginaciju** (`limit/offset`)
- Ispravni status kodovi: 201 create, 204 delete, 400 pravilo, 403 zabranjeno, 404 ne postoji, 409 duplikat, 422 validacija

Zahtjev 6: validacija i error handling

- **Pydantic scheme za svaki input:** zasebne Create, Update i Response varijante - nikad isti DTO za input i output
- Response scheme ne izlažu interne podatke: `hashed_password` nikad ne izlazi iz API-ja
- Validatori za domenska ograničenja: datum u prošlosti, dopuštene vrijednosti, rokovi u ispravnom poretku
- **Standardiziran error format:** svaki error vraća JSON s `code` i `message`, kroz globalni exception handler
- Partial update kroz PATCH s `exclude_unset=True`

422 vraća Pydantic za oblik podataka, 400/409 vraća service za smisao. Na obrani pitam u čemu je razlika.

Zahtjev 7: testovi

Pytest testovi koji dokazuju da pravila vrijede:

Područje	Minimum
Autentikacija	uspješan login, kriva lozinka, istekli token, refresh
Autorizacija	svaka rola: što smije i što dobiva 403
Ownership	korisnik A ne vidi i ne mijenja podatke korisnika B
Poslovna pravila	pravilo prolazi, pravilo zabranjuje, granični slučaj
Constrainti	duplikat vraća 409, nevažeći input 422

Svaki endpoint ima barem jedan **negativni test**. Vremenski ovisna pravila testirajte **freezegunom** - test ne smije ovisiti o danu pokretanja.

Zahtjev 8: Git i predaja

- Projekt je na **GitHubu** od prvog tjedna - repozitorij mi šaljete odmah
- **Commitovi kroz semestar**, ne jedan veliki commit dan prije obrane - povijest commitova je dio ocjene kvalitete
- Smislene commit poruke: „feat: add registration deadline check”, ne „update”
- `.env` nikad u gitu - u repozitoriju stoji `.env.example`
- `README.md` s uputama za pokretanje: baza, migracije, seed, server

Repozitorij s tri commita od kojih je jedan „final version” govori mi više od bilo kojeg odgovora na usmenom.

Bodovna raspodjela 1. dijela (45 bodova)

Komponenta	Bodovi
Modeliranje baze, migracije i seed	6
Autentikacija: bcrypt, JWT, refresh	7
Autorizacija: role i ownership	7
Poslovna pravila	7
API dizajn i status kodovi	5
Validacija i error handling	4
Testovi	5
Arhitektura i kvaliteta koda	4
Ukupno	45

Bodovi vrijede samo uz **razumijevanje**: dio koji na usmenom ne znate objasniti ne nosi bodove, ma koliko dobro radio.

Checklist za predaju 1. dijela

Prije obrane provjerite da sve prolazi na svježem cloneu repozitorija:

- ❶ `docker compose up -d db` podiže bazu
- ❷ `alembic upgrade head` prolazi bez greške
- ❸ `python -m app.seed` puni demo podatke
- ❹ `uvicorn app.main:app` se podiže, `/docs` pokazuje sve endpointe
- ❺ `pytest` je zelen
- ❻ README objašnjava sve gornje korake

„Radi na mom laptopu” nije predaja. Ako ja ne mogu pokrenuti projekt po README-u, obrana počinje s minusom.

Što trebate instalirati

Alat	Verzija	Svrha
Git	najnovija	verzioniranje koda
Python	3.11+	backend (FastAPI)
Docker Desktop	najnovija	PostgreSQL baza
VS Code	najnovija	editor

Node.js za frontend dolazi u 2. dijelu - za backend vam ne treba. Sve se instalira kroz službene stranice ili package managere; detalji na sljedećim slideovima.

Git

```
# Windows - preuzmite installer sa https://git-scm.com
# macOS
brew install git
# Linux (Debian/Ubuntu)
sudo apt install git

# Provjera
git --version

# Osnovna konfiguracija (obavezno nakon instalacije)
git config --global user.name "Ime Prezime"
git config --global user.email "email@example.com"
```

Ako nemate GitHub račun, napravite ga odmah - trebat će vam za predaju projekta i za GitHub Education.

Python i uv

```
# Windows - preuzmite sa https://python.org
# OBAVEZNO oznacite "Add Python to PATH" u installeru

# macOS
brew install python@3.12

# Linux
sudo apt install python3

# uv - moderni alat za pakete i virtualna okruzenja
# Windows: winget install astral-sh.uv
# macOS: brew install uv
# Linux: curl -Lsf https://astral.sh/uv/install.sh | sh

# Provjera
python --version # ili python3 --version
uv --version
```

Virtualna okruženja i pakete vodimo kroz **uv**: uv venv stvara okruženje, uv pip install instalira pakete - brže od klasičnog pipa, a svaki projekt ostaje izoliran od sustava.

Docker Desktop

Preuzmite Docker Desktop s docker.com. Uključuje Docker Engine i Docker Compose.

```
# Provjera nakon instalacije  
docker --version  
docker compose version  
  
# Pokretanje PostgreSQL baze za razvoj  
docker compose up -d db
```

Docker koristimo **samo za bazu** - ne trebate kontejnerizirati svoj backend. PostgreSQL lokalno bez Dockera je moguć, ali Docker Compose je jednostavniji jer konfiguracija ostaje u `docker-compose.yml`.

VS Code i ekstenzije

VS Code preuzmite s `code.visualstudio.com`. Preporučene ekstenzije za backend:

- **Python** (Microsoft) - IntelliSense, debugging, venv podrška
- **SQLTools** + PostgreSQL driver - pregled baze iz editora
- **REST Client** ili **Thunder Client** - testiranje API-ja bez Postmana

Smijete koristiti i drugi editor (PyCharm, IntelliJ), ali upute na predavanjima prate VS Code.

GitHub Education - što dobivate

Kao studenti imate pravo na **GitHub Student Developer Pack** koji uključuje:

- **GitHub Copilot** besplatno - AI autocomplete u editoru
- Pristup **AI modelima** kroz Copilot Chat - najnovije generacije GPT i Claude modela
- GitHub Pro - privatni repozitoriji, Actions minute, Codespaces
- Razni bonusi: domene, cloud krediti, alati

Aktivacija: `education.github.com` → prijavite se s GitHub računom → potvrdite studentski status (index ili potvrda s fakulteta).

GitHub Copilot u VS Code-u

Nakon aktivacije GitHub Education, instalirajte **GitHub Copilot** ekstenziju u VS Code.

Dva načina korištenja:

- 1 **Inline suggestions** - dok tipkate, Copilot predlaže nastavak koda. Tab za prihvatanje, Esc za odbijanje.
- 2 **Copilot Chat** (Ctrl+Shift+I) - razgovor s AI-jem o kodu. Selektirajte dio koda i pitajte za objašnjenje, refactoring ili debugging.

U Copilot Chatu birate model - isprobajte različite, svaki ima drugačije snage.

Copilot Chat - korisne naredbe

Naredba	Što radi
<code>/explain</code>	Objašnjava selektirani kod
<code>/fix</code>	Predlaže popravak za grešku
<code>/tests</code>	Generira testove za selektirani kod
<code>/doc</code>	Generira dokumentaciju
<code>@workspace</code>	Daje kontekst cijelog projekta

Selektirajte funkciju → otvorite Chat → napišite `/explain` ili postavite pitanje prirodnim jezikom. Chat vidi vaš kod i daje kontekstualni odgovor.

Claude Code u VS Code-u

Claude Code je Anthropicov agentski alat koji se integrira u VS Code kao ekstenzija.

Instalacija:

- 1 Napravite račun na `claude.ai`
- 2 U VS Code instalirajte ekstenziju **Claude Code** (Anthropic)
- 3 Prijavite se kroz ekstenziju - otvori se browser za autentikaciju
- 4 Claude se pojavljuje kao panel u VS Code-u

Claude čita vaše datoteke, razumije strukturu projekta i može predlagati promjene direktno u editoru. Za razliku od Copilota koji radi inline, Claude radi u **agentskom** modu - može čitati, pisati i izvršavati naredbe.

Claude vs Copilot - kad koristiti koji

Copilot

- Brzi inline prijedlozi dok tipkate
- Autocomplete za poznate obrasce
- Kratka pitanja o kodu (/explain)
- Generiranje boilerplate koda

Claude Code

- Složeniji zadaci s više datoteka
- Debugging - čita error, traži uzrok
- Razumijevanje arhitekture
- Pisanje i pokretanje testova

Ne trebate birati samo jedan. Copilot za svakodnevno tipkanje, Claude kad trebate razmisliti o problemu ili napraviti veću promjenu.

AI smijete koristiti - uz razumijevanje

AI alati su dio modernog razvoja softvera i zabraniti ih nema smisla. Cilj kolegija nije provjeriti možete li pisati kod iz glave, nego **razumijete li što taj kod radi i zašto**.

Na usmenom ne gledam **jeste li** koristili AI. Gledam **razumijete li** kod koji ste predali. Ako ne možete objasniti vlastiti kod, svejedno je tko ga je napisao - bodove ne nosi.

Kako AI koristiti za učenje

Pristup koji radi:

- 1 **Prvo pokušajte sami** - razmislite o problemu 10-15 minuta prije nego što pitate AI
- 2 **Pitajte za objašnjenje, ne za gotov kod** - „Objasni mi kako radi JWT refresh flow” umjesto „Napiši mi refresh endpoint”
- 3 **Čitajte generirani kod red po red** - što ne razumijete, tražite da objasni
- 4 **Reproducirajte sami** - nakon što razumijete, napišite sličan kod bez AI-ja
- 5 **Testirajte rubne slučajeve** - što ako token istekne? ako korisnik nema pravo?

Što ne raditi: copy-paste cijelog modula bez čitanja, traženje kompletnog featurea odjednom, slijepo ponavljanje „ne radi, popravi” umjesto čitanja error poruke.

Dobar prompt vs loš prompt

Loš prompt

- „Napravi mi login”
- „Ne radi, popravi”
- „Napiši testove”
- „Dodaj autorizaciju”

Dobar prompt

- „Objasni zašto login vraća 401 na ispravan password - evo requesta i responsea”
- „Imam ovaj service - objasni kako ownership check radi”
- „Napiši test koji provjerava da korisnik ne može vidjeti tuđe podatke”

Što precizniji prompt, to bolji odgovor. Dajte kontekst: error poruku, dio koda, što ste pokušali.

1. obrana (16.11.): kako izgleda

Demonstracija API-ja kroz Swagger, pregled repozitorija i ključnih dijelova koda, pitanja o odlukama i edge caseovima.

Tipična pitanja:

- Objasni arhitekturu - što radi router, što service, što repository?
- Kako radi JWT flow? Gdje se hashira lozinka?
- Gdje provjeravaš ownership? Pokaži mi liniju koda.
- Kako backend zabranjuje akciju nakon isteka roka?
- Zašto ovaj endpoint vraća 403, a ne 404?
- Što testira ovaj test i što bi se pokvarilo da ga nema?

Na kraju: **mini promjena uživo** - dodaj polje na model, napravi migraciju, prilagodi schemu i endpoint.

Što je dovoljan odgovor na usmenom

Ne tražim savršene odgovore. Tražim da **razumijete** svoj kod.

Prolazi

- „Dependency čita JWT iz headera, dekodira ga i vraća usera - endpoint ga dobiva kroz Depends”
- „403 jer je autenticiran, ali nema pravo na taj resurs”
- „Test dokazuje da korisnik ne može mijenjati tuđi zapis”

Ne prolazi

- „To mi je AI napisao, ne znam točno”
- „Mislim da to radi nešto s tokenima”
- „Nisam siguran zašto je 403, a ne 401”

Rokovi i sljedeći koraci

Što	Kada
Odabir teme i repozitorij na GitHubu	do 12.10.
1. obrana: backend	16.11.
Prezentacija „Projekt 2” (frontend zahtjevi)	23.11.
2. obrana: full-stack + deploy	18.01.
Rezervni termin predaje	veljački rokovi

Počnite **ovaj tjedan**: instalirajte okruženje, odaberite temu, napravite repozitorij. Svako predavanje reproducirajte na svom projektu - tko prati ritam predavanja, backend mu je gotov prije roka.

Pitanja?